



Lecture 10

Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

August 11, 2026

Constructing Parse table

Constructing Parse table

- Table can be constructed if for every non terminal, every lookahead symbol can be handled by at most one production

Constructing Parse table

- Table can be constructed if for every non terminal, every lookahead symbol can be handled by at most one production
- **First(a)** for a string of terminals and non terminals a is Set of symbols that might begin the fully expanded (made of only tokens) version of a

Constructing Parse table

- Table can be constructed if for every non terminal, every lookahead symbol can be handled by at most one production
- **First(a)** for a string of terminals and non terminals a is Set of symbols that might begin the fully expanded (made of only tokens) version of a
- **Follow(X)** for a non terminal X is set of symbols that might follow the derivation of X in the input stream

Computation of FIRST set

Computation of FIRST set

- If X is a terminal symbol then $First(X) = X$

Computation of FIRST set

- If X is a terminal symbol then $First(X) = X$
- If $X \rightarrow \epsilon$ is a production then ϵ is in $First(X)$.

Computation of FIRST set

- If X is a terminal symbol then $First(X) = X$
- If $X \rightarrow \epsilon$ is a production then ϵ is in $First(X)$.
- If X is a non terminal and $X \rightarrow Y_1 Y_2 \cdots Y_k$ is a production then if for some " i ", " a " is in $First(Y_i)$ and ϵ is in all of $First(Y_j)$ (such that $j < i$) then a is in $First(X)$

Computation of FIRST set

- If X is a terminal symbol then $First(X) = X$
- If $X \rightarrow \epsilon$ is a production then ϵ is in $First(X)$.
- If X is a non terminal and $X \rightarrow Y_1 Y_2 \cdots Y_k$ is a production then if for some " i ", " a " is in $First(Y_i)$ and ϵ is in all of $First(Y_j)$ (such that $j < i$) then a is in $First(X)$
- If ϵ is in $First(Y_1) \cdots First(Y_k)$ then ϵ is in $First(X)$

Example

- For the expression grammar

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | id$$

Example

- For the expression grammar

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | id$$

- $First(E) = First(T) = First(F) = (, id$

Example

- For the expression grammar

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | id$$

- $First(E) = First(T) = First(F) = (, id$
- $First(E') = +, \epsilon$

Example

- For the expression grammar

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | id$$

- $First(E) = First(T) = First(F) = (, id$
- $First(E') = +, \epsilon$
- $First(T') = *, \epsilon$

Compute follow sets

Compute follow sets

- Place \$ in $\text{follow}(S)$

Compute follow sets

- Place $\$$ in $\text{follow}(S)$
- If there is a production $A \rightarrow \alpha B \beta$

Compute follow sets

- Place $\$$ in $follow(S)$
- If there is a production $A \rightarrow \alpha B \beta$ then everything in $first(\beta)$ (except ϵ) is in $follow(B)$

Compute follow sets

- Place $\$$ in $follow(S)$
- If there is a production $A \rightarrow \alpha B \beta$ then everything in $first(\beta)$ (except ϵ) is in $follow(B)$
- If there is a production $A \rightarrow \alpha B$

Compute follow sets

- Place $\$$ in $follow(S)$
- If there is a production $A \rightarrow \alpha B \beta$ then everything in $first(\beta)$ (except ϵ) is in $follow(B)$
- If there is a production $A \rightarrow \alpha B$ then everything in $follow(A)$ is in $follow(B)$

Compute follow sets

- Place \$ in $follow(S)$
- If there is a production $A \rightarrow \alpha B \beta$ then everything in $first(\beta)$ (except ϵ) is in $follow(B)$
- If there is a production $A \rightarrow \alpha B$ then everything in $follow(A)$ is in $follow(B)$
- If there is a production $A \rightarrow \alpha B \beta$ and $First(\beta)$ contains ϵ then everything in $follow(A)$ is in $follow(B)$

Example

- For the expression grammar

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | id$$

Example

- For the expression grammar

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | id$$

- $follow(E) = follow(E') = \$,)$

Example

- For the expression grammar

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | id$$

- $follow(E) = follow(E') = \$,)$
- $follow(T) = follow(T') = \$,), +$

Example

- For the expression grammar

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | id$$

- $follow(E) = follow(E') = \$,)$
- $follow(T) = follow(T') = \$,), +$
- $follow(F) = \$,), +, *$

Example

- For the expression grammar

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \epsilon$$

$$F \rightarrow (E) | id$$

- $follow(E) = follow(E') = \$,)$
- $follow(T) = follow(T') = \$,), +$
- $follow(F) = \$,), +, *$

Construction of parse table

Construction of parse table

- for each production $A \rightarrow \alpha$ do

Construction of parse table

- for each production $A \rightarrow \alpha$ do
 - ▶ for each terminal 'a' in $first(\alpha)$
 $M[A, a] = A \rightarrow \alpha$

Construction of parse table

- for each production $A \rightarrow \alpha$ do
 - ▶ for each terminal 'a' in $first(\alpha)$
 $M[A, a] = A \rightarrow \alpha$
 - ▶ If ϵ is in $First(\alpha)$
 $M[A, b] = A \rightarrow \alpha$
for each terminal b in $follow(A)$

Construction of parse table

- for each production $A \rightarrow \alpha$ do
 - ▶ for each terminal 'a' in $first(\alpha)$
 $M[A, a] = A \rightarrow \alpha$
 - ▶ If ϵ is in $First(\alpha)$
 $M[A, b] = A \rightarrow \alpha$
for each terminal b in $follow(A)$
 - ▶ If ϵ is in $First(a)$ and $\$$ is in $follow(A)$
 $M[A, \$] = A \rightarrow \alpha$

Construction of parse table

- for each production $A \rightarrow \alpha$ do
 - ▶ for each terminal 'a' in $first(\alpha)$
 $M[A, a] = A \rightarrow \alpha$
 - ▶ If ϵ is in $First(\alpha)$
 $M[A, b] = A \rightarrow \alpha$
for each terminal b in $follow(A)$
 - ▶ If ϵ is in $First(a)$ and $\$$ is in $follow(A)$
 $M[A, \$] = A \rightarrow \alpha$
- A grammar whose parse table has no multiple entries is called $LL(1)$

Error handling

Error handling

- Stop at the first error and print a message

Error handling

- Stop at the first error and print a message
 - ▶ Compiler writer friendly

Error handling

- Stop at the first error and print a message
 - ▶ Compiler writer friendly
 - ▶ But not user friendly

Error handling

- Stop at the first error and print a message
 - ▶ Compiler writer friendly
 - ▶ But not user friendly
- Every reasonable compiler must recover from errors and identify as many errors as possible

Error handling

- Stop at the first error and print a message
 - ▶ Compiler writer friendly
 - ▶ But not user friendly
- Every reasonable compiler must recover from errors and identify as many errors as possible
- However, multiple error messages due to a single fault must be avoided

Error handling

- Stop at the first error and print a message
 - ▶ Compiler writer friendly
 - ▶ But not user friendly
- Every reasonable compiler must recover from errors and identify as many errors as possible
- However, multiple error messages due to a single fault must be avoided
- Error recovery methods

Error handling

- Stop at the first error and print a message
 - ▶ Compiler writer friendly
 - ▶ But not user friendly
- Every reasonable compiler must recover from errors and identify as many errors as possible
- However, multiple error messages due to a single fault must be avoided
- Error recovery methods
 - ▶ Panic model

Error handling

- Stop at the first error and print a message
 - ▶ Compiler writer friendly
 - ▶ But not user friendly
- Every reasonable compiler must recover from errors and identify as many errors as possible
- However, multiple error messages due to a single fault must be avoided
- Error recovery methods
 - ▶ Panic model
 - ▶ Phase level recovery

Error handling

- Stop at the first error and print a message
 - ▶ Compiler writer friendly
 - ▶ But not user friendly
- Every reasonable compiler must recover from errors and identify as many errors as possible
- However, multiple error messages due to a single fault must be avoided
- Error recovery methods
 - ▶ Panic model
 - ▶ Phase level recovery
 - ▶ Error production

Error handling

- Stop at the first error and print a message
 - ▶ Compiler writer friendly
 - ▶ But not user friendly
- Every reasonable compiler must recover from errors and identify as many errors as possible
- However, multiple error messages due to a single fault must be avoided
- Error recovery methods
 - ▶ Panic model
 - ▶ Phase level recovery
 - ▶ Error production
 - ▶ Global correction

Panic Model

- Simplest and the most popular method

Panic Model

- Simplest and the most popular method
- Most tools provide for specifying panic mode recovery in the grammar

Panic Model

- Simplest and the most popular method
- Most tools provide for specifying panic mode recovery in the grammar
- When an error is detected

Panic Model

- Simplest and the most popular method
- Most tools provide for specifying panic mode recovery in the grammar
- When an error is detected
 - ▶ Discard tokens one at a time until a set of tokens is found whose role is clear

Panic Model

- Simplest and the most popular method
- Most tools provide for specifying panic mode recovery in the grammar
- When an error is detected
 - ▶ Discard tokens one at a time until a set of tokens is found whose role is clear
 - ▶ Skip to the next token that can be placed reliably in the parse tree

Panic Model

- Simplest and the most popular method
- Most tools provide for specifying panic mode recovery in the grammar
- When an error is detected
 - ▶ Discard tokens one at a time until a set of tokens is found whose role is clear
 - ▶ Skip to the next token that can be placed reliably in the parse tree
- Consider following code

$a = b + c;$

$x = p \quad r;$

$h = x < 0;$

Panic Model

- Simplest and the most popular method
- Most tools provide for specifying panic mode recovery in the grammar
- When an error is detected
 - ▶ Discard tokens one at a time until a set of tokens is found whose role is clear
 - ▶ Skip to the next token that can be placed reliably in the parse tree
- Consider following code
$$a = b + c;$$
$$x = p \quad r;$$
$$h = x < 0;$$
- Panic mode recovery for block skip ahead to next ';' and try to parse the next expression

Panic Model

- Simplest and the most popular method
- Most tools provide for specifying panic mode recovery in the grammar
- When an error is detected
 - ▶ Discard tokens one at a time until a set of tokens is found whose role is clear
 - ▶ Skip to the next token that can be placed reliably in the parse tree
- Consider following code
$$a = b + c;$$
$$x = p \quad r;$$
$$h = x < 0;$$
- Panic mode recovery for block skip ahead to next ';' and try to parse the next expression
- It discards one expression and tries to continue parsing

Panic Model

- Simplest and the most popular method
- Most tools provide for specifying panic mode recovery in the grammar
- When an error is detected
 - ▶ Discard tokens one at a time until a set of tokens is found whose role is clear
 - ▶ Skip to the next token that can be placed reliably in the parse tree
- Consider following code
$$a = b + c;$$
$$x = p \quad r;$$
$$h = x < 0;$$
- Panic mode recovery for block skip ahead to next ';' and try to parse the next expression
- It discards one expression and tries to continue parsing
- May fail if no further ';' is found

Panic Model

- Simplest and the most popular method
- Most tools provide for specifying panic mode recovery in the grammar
- When an error is detected
 - ▶ Discard tokens one at a time until a set of tokens is found whose role is clear
 - ▶ Skip to the next token that can be placed reliably in the parse tree
- Consider following code
$$a = b + c;$$
$$x = p \quad r;$$
$$h = x < 0;$$
- Panic mode recovery for block skip ahead to next ';' and try to parse the next expression
- It discards one expression and tries to continue parsing
- May fail if no further ';' is found

Phase Level Recovery

Phase Level Recovery

- Make local correction to the input

Phase Level Recovery

- Make local correction to the input
- Works only in limited situations

Phase Level Recovery

- Make local correction to the input
- Works only in limited situations
 - ▶ A common programming error which is easily detected

Phase Level Recovery

- Make local correction to the input
- Works only in limited situations
 - ▶ A common programming error which is easily detected
 - ▶ For example insert a “;” after closing “}” of a class definition

Phase Level Recovery

- Make local correction to the input
- Works only in limited situations
 - ▶ A common programming error which is easily detected
 - ▶ For example insert a “;” after closing “}” of a class definition
 - ▶ Does not work very well!

Error Production

Error Production

- Add erroneous constructs as productions in the grammar

Error Production

- Add erroneous constructs as productions in the grammar
- Works only for most common mistakes which can be easily identified

Error Production

- Add erroneous constructs as productions in the grammar
- Works only for most common mistakes which can be easily identified
- Essentially makes common errors as part of the grammar

Error Production

- Add erroneous constructs as productions in the grammar
- Works only for most common mistakes which can be easily identified
- Essentially makes common errors as part of the grammar
- Complicates the grammar and does not work very well

Global Correction

Global Correction

- Considering the program as a whole find a correct “nearby” program

Global Correction

- Considering the program as a whole find a correct “nearby” program
- Nearness may be measured using certain metric

Global Correction

- Considering the program as a whole find a correct “nearby” program
- Nearness may be measured using certain metric
- PL/C compiler implemented this scheme: anything could be compiled!

Global Correction

- Considering the program as a whole find a correct “nearby” program
- Nearness may be measured using certain metric
- PL/C compiler implemented this scheme: anything could be compiled!
- It is complicated and not a very good idea!

Error Recovery in LL(1) parser

Error Recovery in LL(1) parser

- Error occurs when a parse table entry $M[A, a]$ is empty

Error Recovery in LL(1) parser

- Error occurs when a parse table entry $M[A, a]$ is empty
- Skip symbols in the input until a token in a selected set (synch) appears

Error Recovery in LL(1) parser

- Error occurs when a parse table entry $M[A, a]$ is empty
- Skip symbols in the input until a token in a selected set (synch) appears
- Place symbols in $\text{follow}(A)$ in synch set. Skip tokens until an element in $\text{follow}(A)$ is seen. Pop(A) and continue parsing

Error Recovery in LL(1) parser

- Error occurs when a parse table entry $M[A, a]$ is empty
- Skip symbols in the input until a token in a selected set (synch) appears
- Place symbols in $\text{follow}(A)$ in synch set. Skip tokens until an element in $\text{follow}(A)$ is seen. Pop(A) and continue parsing
- Add symbol in $\text{first}(A)$ in synch set. Then it may be possible to resume parsing according to A if a symbol in $\text{first}(A)$ appears in input.

Error Recovery in LL(1) parser

- Error occurs when a parse table entry $M[A, a]$ is empty
- Skip symbols in the input until a token in a selected set (synch) appears
- Place symbols in $\text{follow}(A)$ in synch set. Skip tokens until an element in $\text{follow}(A)$ is seen. Pop(A) and continue parsing
- Add symbol in $\text{first}(A)$ in synch set. Then it may be possible to resume parsing according to A if a symbol in $\text{first}(A)$ appears in input.

Bottom up parsing

Bottom up parsing

- Construct a parse tree for an input string beginning at leaves and going towards root

Bottom up parsing

- Construct a parse tree for an input string beginning at leaves and going towards root
- Reduce a string w of input to start symbol of grammar

Bottom up parsing

- Construct a parse tree for an input string beginning at leaves and going towards root
- Reduce a string w of input to start symbol of grammar
- Consider the following grammar

$$S \rightarrow aABe$$

$$A \rightarrow Abc|b$$

$$B \rightarrow d$$

Bottom up parsing

- Construct a parse tree for an input string beginning at leaves and going towards root
- Reduce a string w of input to start symbol of grammar
- Consider the following grammar

$$S \rightarrow aABe$$

$$A \rightarrow Abc|b$$

$$B \rightarrow d$$

Reduction of the string

a b b c d e

Bottom up parsing

- Construct a parse tree for an input string beginning at leaves and going towards root
- Reduce a string w of input to start symbol of grammar
- Consider the following grammar

$$S \rightarrow aABe$$

$$A \rightarrow Abc|b$$

$$B \rightarrow d$$

Reduction of the string

a b b c d e

a A b c d e

Bottom up parsing

- Construct a parse tree for an input string beginning at leaves and going towards root
- Reduce a string w of input to start symbol of grammar
- Consider the following grammar

$$S \rightarrow aABe$$

$$A \rightarrow Abc|b$$

$$B \rightarrow d$$

Reduction of the string

a b b c d e

a A b c d e

a A d e

Bottom up parsing

- Construct a parse tree for an input string beginning at leaves and going towards root
- Reduce a string w of input to start symbol of grammar
- Consider the following grammar

$$S \rightarrow aABe$$

$$A \rightarrow Abc|b$$

$$B \rightarrow d$$

Reduction of the string

a b b c d e

a A b c d e

a A d e

a A B e

Bottom up parsing

- Construct a parse tree for an input string beginning at leaves and going towards root
- Reduce a string w of input to start symbol of grammar
- Consider the following grammar

$$S \rightarrow aABe$$

$$A \rightarrow Abc|b$$

$$B \rightarrow d$$

Reduction of the string

a b b c d e

a A b c d e

a A d e

a A B e

S

Bottom up parsing

- Construct a parse tree for an input string beginning at leaves and going towards root
- Reduce a string w of input to start symbol of grammar
- Consider the following grammar

$$S \rightarrow aABe$$

$$A \rightarrow Abc|b$$

$$B \rightarrow d$$

Reduction of the string

a b b c d e

a A b c d e

a A d e

a A B e

S

Rightmost derivation

Bottom up parsing

- Construct a parse tree for an input string beginning at leaves and going towards root
- Reduce a string w of input to start symbol of grammar
- Consider the following grammar

$$S \rightarrow aABe$$

$$A \rightarrow Abc|b$$

$$B \rightarrow d$$

Reduction of the string

a b b c d e

a A b c d e

a A d e

a A B e

S

Rightmost derivation

$$S \rightarrow aABe$$

Bottom up parsing

- Construct a parse tree for an input string beginning at leaves and going towards root
- Reduce a string w of input to start symbol of grammar
- Consider the following grammar

$$S \rightarrow aABe$$

$$A \rightarrow Abc|b$$

$$B \rightarrow d$$

Reduction of the string

a b b c d e

a A b c d e

a A d e

a A B e

S

Rightmost derivation

$$S \rightarrow aABe$$

$$S \rightarrow aAde$$

Bottom up parsing

- Construct a parse tree for an input string beginning at leaves and going towards root
- Reduce a string w of input to start symbol of grammar
- Consider the following grammar

$$S \rightarrow aABe$$

$$A \rightarrow Abc|b$$

$$B \rightarrow d$$

Reduction of the string

a b b c d e

a A b c d e

a A d e

a A B e

S

Rightmost derivation

$$S \rightarrow aABe$$

$$S \rightarrow aAde$$

$$S \rightarrow aAbcde$$

Bottom up parsing

- Construct a parse tree for an input string beginning at leaves and going towards root
- Reduce a string w of input to start symbol of grammar
- Consider the following grammar

$$S \rightarrow aABe$$

$$A \rightarrow Abc|b$$

$$B \rightarrow d$$

Reduction of the string

a b b c d e

a A b c d e

a A d e

a A B e

S

Rightmost derivation

$$S \rightarrow aABe$$

$$S \rightarrow aAde$$

$$S \rightarrow aAbcde$$

$$S \rightarrow abbcde$$

Shift reduce parsing

Shift reduce parsing

- Split string being parsed into two parts

Shift reduce parsing

- Split string being parsed into two parts
 - ▶ Two parts are separated by a special character "."

Shift reduce parsing

- Split string being parsed into two parts
 - ▶ Two parts are separated by a special character "."
 - ▶ Left part is a string of terminals and non terminals

Shift reduce parsing

- Split string being parsed into two parts
 - ▶ Two parts are separated by a special character "."
 - ▶ Left part is a string of terminals and non terminals
 - ▶ Right part is a string of terminals

Shift reduce parsing

- Split string being parsed into two parts
 - ▶ Two parts are separated by a special character "."
 - ▶ Left part is a string of terminals and non terminals
 - ▶ Right part is a string of terminals
- Initially the input is Bottom up parsing .w

Shift reduce parsing

- Split string being parsed into two parts
 - ▶ Two parts are separated by a special character "."
 - ▶ Left part is a string of terminals and non terminals
 - ▶ Right part is a string of terminals
- Initially the input is Bottom up parsing .w
- Bottom up parsing has two actions

Shift reduce parsing

- Split string being parsed into two parts
 - ▶ Two parts are separated by a special character "."
 - ▶ Left part is a string of terminals and non terminals
 - ▶ Right part is a string of terminals
- Initially the input is Bottom up parsing $.w$
- Bottom up parsing has two actions
- Shift: move terminal symbol from right string to left string if string before shift is then string after shift is $\alpha.pqr \rightarrow \alpha p.qr$

Shift reduce parsing

- Split string being parsed into two parts
 - ▶ Two parts are separated by a special character "."
 - ▶ Left part is a string of terminals and non terminals
 - ▶ Right part is a string of terminals
- Initially the input is Bottom up parsing $.w$
- Bottom up parsing has two actions
- Shift: move terminal symbol from right string to left string if string before shift is then string after shift is $\alpha.pqr$ $\alpha p.qr$
- Reduce: immediately on the left of "." identify a string same as RHS of a production and replace it by LHS if string before reduce action is $\alpha\beta.pqr$ and $A \rightarrow \beta$ is a production then string after reduction is $\alpha A.pqr$

Example

Example

- Assume grammar is $E \rightarrow E + E \mid E * E \mid id$

Example

- Assume grammar is $E \rightarrow E + E \mid E * E \mid id$
- Parse $id * id + id$

Example

- Assume grammar is $E \rightarrow E + E | E * E | id$
- Parse $id * id + id$

String	Action
--------	--------

Example

- Assume grammar is $E \rightarrow E + E | E * E | id$
- Parse $id * id + id$

String	Action
.id*id+id	shift

Example

- Assume grammar is $E \rightarrow E + E | E * E | id$
- Parse $id * id + id$

String	Action
.id*id+id	shift
id.*id+id	reduce $E \rightarrow id$

Example

- Assume grammar is $E \rightarrow E + E | E * E | id$
- Parse $id * id + id$

String	Action
.id*id+id	shift
id.*id+id	reduce $E \rightarrow id$
E.*id+id	shift

Example

- Assume grammar is $E \rightarrow E + E | E * E | id$
- Parse $id * id + id$

String	Action
.id*id+id	shift
id.*id+id	reduce $E \rightarrow id$
E.*id+id	shift
E*.id+id	shift

Example

- Assume grammar is $E \rightarrow E + E | E * E | id$
- Parse $id * id + id$

String	Action
.id*id+id	shift
id.*id+id	reduce $E \rightarrow id$
E.*id+id	shift
E*.id+id	shift
E*id.+id	reduce $E \rightarrow id$

Example

- Assume grammar is $E \rightarrow E + E \mid E * E \mid id$
- Parse $id * id + id$

String	Action
.id*id+id	shift
id.*id+id	reduce $E \rightarrow id$
E.*id+id	shift
E*.id+id	shift
E*id.+id	reduce $E \rightarrow id$
E*E.+id	reduce $E \rightarrow E * E$

Example

- Assume grammar is $E \rightarrow E + E | E * E | id$
- Parse $id * id + id$

String	Action
.id*id+id	shift
id.*id+id	reduce $E \rightarrow id$
E.*id+id	shift
E*.id+id	shift
E*id.+id	reduce $E \rightarrow id$
E*E.+id	reduce $E \rightarrow E * E$
E.+id	shift

Example

- Assume grammar is $E \rightarrow E + E | E * E | id$
- Parse $id * id + id$

String	Action
.id*id+id	shift
id.*id+id	reduce $E \rightarrow id$
E.*id+id	shift
E*.id+id	shift
E*id.+id	reduce $E \rightarrow id$
E*E.+id	reduce $E \rightarrow E * E$
E.+id	shift
E+.id	shift

Example

- Assume grammar is $E \rightarrow E + E | E * E | id$
- Parse $id * id + id$

String	Action
.id*id+id	shift
id.*id+id	reduce $E \rightarrow id$
E.*id+id	shift
E*.id+id	shift
E*id.+id	reduce $E \rightarrow id$
E*E.+id	reduce $E \rightarrow E * E$
E.+id	shift
E+.id	shift
E+id.	reduce $E \rightarrow id$

Example

- Assume grammar is $E \rightarrow E + E | E * E | id$
- Parse $id * id + id$

String	Action
.id*id+id	shift
id.*id+id	reduce $E \rightarrow id$
E.*id+id	shift
E*.id+id	shift
E*id.+id	reduce $E \rightarrow id$
E*E.+id	reduce $E \rightarrow E * E$
E.+id	shift
E+.id	shift
E+id.	reduce $E \rightarrow id$
E+E.	reduce $E \rightarrow E + E$

Example

- Assume grammar is $E \rightarrow E + E | E * E | id$
- Parse $id * id + id$

String	Action
.id*id+id	shift
id.*id+id	reduce $E \rightarrow id$
E.*id+id	shift
E*.id+id	shift
E*id.+id	reduce $E \rightarrow id$
E*E.+id	reduce $E \rightarrow E * E$
E.+id	shift
E+.id	shift
E+id.	reduce $E \rightarrow id$
E+E.	reduce $E \rightarrow E + E$
E.	ACCEPT

Shift reduce parsing

Shift reduce parsing

- Symbols on the left of "." are kept on a stack

Shift reduce parsing

- Symbols on the left of "." are kept on a stack
 - ▶ Top of the stack is at "."

Shift reduce parsing

- Symbols on the left of "." are kept on a stack
 - ▶ Top of the stack is at "."
 - ▶ Shift pushes a terminal on the stack

Shift reduce parsing

- Symbols on the left of "." are kept on a stack
 - ▶ Top of the stack is at "."
 - ▶ Shift pushes a terminal on the stack
 - ▶ Reduce pops symbols (rhs of production) and pushes a non terminal (lhs of production) onto the stack

Shift reduce parsing

- Symbols on the left of "." are kept on a stack
 - ▶ Top of the stack is at "."
 - ▶ Shift pushes a terminal on the stack
 - ▶ Reduce pops symbols (rhs of production) and pushes a non terminal (lhs of production) onto the stack
- The most important issue: when to shift and when to reduce

Shift reduce parsing

- Symbols on the left of "." are kept on a stack
 - ▶ Top of the stack is at "."
 - ▶ Shift pushes a terminal on the stack
 - ▶ Reduce pops symbols (rhs of production) and pushes a non terminal (lhs of production) onto the stack
- The most important issue: when to shift and when to reduce
- Reduce action should be taken only if the result can be reduced to the start symbol

Bottom up parsing

- A more powerful parsing technique

Bottom up parsing

- A more powerful parsing technique
- LR grammars – more expensive than LL

Bottom up parsing

- A more powerful parsing technique
- LR grammars – more expensive than LL
- Can handle left recursive grammars

Bottom up parsing

- A more powerful parsing technique
- LR grammars – more expensive than LL
- Can handle left recursive grammars
- Can handle virtually all the programming languages

Bottom up parsing

- A more powerful parsing technique
- LR grammars – more expensive than LL
- Can handle left recursive grammars
- Can handle virtually all the programming languages
- Natural expression of programming language syntax

Bottom up parsing

- A more powerful parsing technique
- LR grammars – more expensive than LL
- Can handle left recursive grammars
- Can handle virtually all the programming languages
- Natural expression of programming language syntax
- Automatic generation of parsers (Yacc, Bison etc.)

Bottom up parsing

- A more powerful parsing technique
- LR grammars – more expensive than LL
- Can handle left recursive grammars
- Can handle virtually all the programming languages
- Natural expression of programming language syntax
- Automatic generation of parsers (Yacc, Bison etc.)
- Detects errors as soon as possible

Bottom up parsing

- A more powerful parsing technique
- LR grammars – more expensive than LL
- Can handle left recursive grammars
- Can handle virtually all the programming languages
- Natural expression of programming language syntax
- Automatic generation of parsers (Yacc, Bison etc.)
- Detects errors as soon as possible
- Allows better error recovery

Issues in bottom up parsing

Issues in bottom up parsing

- How do we know which action to take

Issues in bottom up parsing

- How do we know which action to take
 - ▶ whether to shift or reduce

Issues in bottom up parsing

- How do we know which action to take
 - ▶ whether to shift or reduce
 - ▶ which production to use for reduction?

Issues in bottom up parsing

- How do we know which action to take
 - ▶ whether to shift or reduce
 - ▶ which production to use for reduction?
- Sometimes parser can reduce but it should not: $X \rightarrow \epsilon$ can always be reduced!

Issues in bottom up parsing

- How do we know which action to take
 - ▶ whether to shift or reduce
 - ▶ which production to use for reduction?
- Sometimes parser can reduce but it should not: $X \rightarrow \epsilon$ can always be reduced!
- Given stack δ and input symbol ' a ', should the parser

Issues in bottom up parsing

- How do we know which action to take
 - ▶ whether to shift or reduce
 - ▶ which production to use for reduction?
- Sometimes parser can reduce but it should not: $X \rightarrow \epsilon$ can always be reduced!
- Given stack δ and input symbol 'a', should the parser
 - ▶ Shift a onto stack (making it δa)

Issues in bottom up parsing

- How do we know which action to take
 - ▶ whether to shift or reduce
 - ▶ which production to use for reduction?
- Sometimes parser can reduce but it should not: $X \rightarrow \epsilon$ can always be reduced!
- Given stack δ and input symbol ' a ', should the parser
 - ▶ Shift a onto stack (making it δa)
 - ▶ Reduce by some production $A \rightarrow \beta$ assuming that stack has form $a\beta$ (making it aA)

Issues in bottom up parsing

- How do we know which action to take
 - ▶ whether to shift or reduce
 - ▶ which production to use for reduction?
- Sometimes parser can reduce but it should not: $X \rightarrow \epsilon$ can always be reduced!
- Given stack δ and input symbol ' a ', should the parser
 - ▶ Shift a onto stack (making it δa)
 - ▶ Reduce by some production $A \rightarrow \beta$ assuming that stack has form $a\beta$ (making it aA)
 - ▶ How to keep track of length of β ?

Handle

Handle

- A string that matches right hand side of a production and whose replacement gives a step in the reverse right most derivation

Handle

- A string that matches right hand side of a production and whose replacement gives a step in the reverse right most derivation
- If $S \rightarrow^{rm*} aAw \rightarrow^{rm} a\beta w$ then β (corresponding to production $A \rightarrow \beta$) in the position following 'a' is a handle of $a\beta w$. The string w consists of only terminal symbols

Handle

- A string that matches right hand side of a production and whose replacement gives a step in the reverse right most derivation
- If $S \rightarrow^{rm*} aAw \rightarrow^{rm} a\beta w$ then β (corresponding to production $A \rightarrow \beta$) in the position following 'a' is a handle of $a\beta w$. The string w consists of only terminal symbols
- We only want to reduce handle and not any rhs

Handle

- A string that matches right hand side of a production and whose replacement gives a step in the reverse right most derivation
- If $S \rightarrow^{rm*} aAw \rightarrow^{rm} a\beta w$ then β (corresponding to production $A \rightarrow \beta$) in the position following 'a' is a handle of $a\beta w$. The string w consists of only terminal symbols
- We only want to reduce handle and not any rhs
- **Handle pruning:** If β is a handle and $A \rightarrow \beta$ is a production then replace β by A

Handle

- A string that matches right hand side of a production and whose replacement gives a step in the reverse right most derivation
- If $S \xrightarrow{rm*} aAw \xrightarrow{rm} a\beta w$ then β (corresponding to production $A \rightarrow \beta$) in the position following 'a' is a handle of $a\beta w$. The string w consists of only terminal symbols
- We only want to reduce handle and not any rhs
- **Handle pruning:** If β is a handle and $A \rightarrow \beta$ is a production then replace β by A
- A right most derivation in reverse can be obtained by handle pruning

Handle

- A string that matches right hand side of a production and whose replacement gives a step in the reverse right most derivation
- If $S \rightarrow^{rm*} aAw \rightarrow^{rm} a\beta w$ then β (corresponding to production $A \rightarrow \beta$) in the position following 'a' is a handle of $a\beta w$. The string w consists of only terminal symbols
- We only want to reduce handle and not any rhs
- **Handle pruning:** If β is a handle and $A \rightarrow \beta$ is a production then replace β by A
- A right most derivation in reverse can be obtained by handle pruning
- Handles always appear at the top of the stack and never inside it. This makes stack a suitable data structure

Handle

- A string that matches right hand side of a production and whose replacement gives a step in the reverse right most derivation
- If $S \rightarrow^{rm*} aAw \rightarrow^{rm} a\beta w$ then β (corresponding to production $A \rightarrow \beta$) in the position following 'a' is a handle of $a\beta w$. The string w consists of only terminal symbols
- We only want to reduce handle and not any rhs
- **Handle pruning:** If β is a handle and $A \rightarrow \beta$ is a production then replace β by A
- A right most derivation in reverse can be obtained by handle pruning
- Handles always appear at the top of the stack and never inside it. This makes stack a suitable data structure
- Bottom up parsing is based on recognizing handles

Conflicts

Conflicts

- The general shift-reduce technique is:

Conflicts

- The general shift-reduce technique is:
 - ▶ if there is no handle on the stack then shift

Conflicts

- The general shift-reduce technique is:
 - ▶ if there is no handle on the stack then shift
 - ▶ If there is a handle then reduce

Conflicts

- The general shift-reduce technique is:
 - ▶ if there is no handle on the stack then shift
 - ▶ If there is a handle then reduce
- However, what happens when there is a choice

Conflicts

- The general shift-reduce technique is:
 - ▶ if there is no handle on the stack then shift
 - ▶ If there is a handle then reduce
- However, what happens when there is a choice
 - ▶ What action to take in case both shift and reduce are valid. **Shift-Reduce conflict**

Conflicts

- The general shift-reduce technique is:
 - ▶ if there is no handle on the stack then shift
 - ▶ If there is a handle then reduce
- However, what happens when there is a choice
 - ▶ What action to take in case both shift and reduce are valid. **Shift-Reduce conflict**
 - ▶ Which rule to use for reduction if reduction is possible by more than one rule?
Reduce-Reduce conflict

Conflicts

- The general shift-reduce technique is:
 - ▶ if there is no handle on the stack then shift
 - ▶ If there is a handle then reduce
- However, what happens when there is a choice
 - ▶ What action to take in case both shift and reduce are valid. **Shift-Reduce conflict**
 - ▶ Which rule to use for reduction if reduction is possible by more than one rule?
Reduce-Reduce conflict
- Conflicts come either because of ambiguous grammars or parsing method is not powerful enough

Shift-Reduce conflict

Shift-Reduce conflict

- Consider the grammar $E \rightarrow E + E \mid E * E \mid id$

Shift-Reduce conflict

- Consider the grammar $E \rightarrow E + E \mid E * E \mid id$
- input $id + id * id$

Shift-Reduce conflict

- Consider the grammar $E \rightarrow E + E \mid E * E \mid id$
- input $id + id * id$

stack	input	action
E+E	*id	reduce $E \rightarrow E + E$
E	*id	shift
E*	id	shift
E*id		reduce $E \rightarrow id$
E*E		reduce $E \rightarrow E * E$
E		

Shift-Reduce conflict

- Consider the grammar $E \rightarrow E + E \mid E * E \mid id$
- input $id + id * id$

stack	input	action
E+E	*id	reduce $E \rightarrow E + E$
E	*id	shift
E*	id	shift
E*id		reduce $E \rightarrow id$
E*E		reduce $E \rightarrow E * E$
E		

stack	input	action
E+E	*id	shift
E+E*	id	shift
E+E*id		reduce $E \rightarrow id$
E+E*E		reduce $E \rightarrow E * E$
E+E		reduce $E \rightarrow E + E$
E		

Reduce-Reduce conflict

Reduce-Reduce conflict

- Consider the grammar $M \rightarrow R + R \mid R + c \mid R$
 $R \rightarrow c$

Reduce-Reduce conflict

- Consider the grammar $M \rightarrow R + R \mid R + c \mid R$
 $R \rightarrow c$
- input $c + c$

Reduce-Reduce conflict

- Consider the grammar $M \rightarrow R + R \mid R + c \mid R$

$$R \rightarrow c$$

- input $c + c$

stack	input	action
	$c+c$	shift
c	$+c$	reduce $R \rightarrow c$
R	$+c$	shift
$R+$	c	shift
$R+c$		reduce $R \rightarrow c$
$R+R$		reduce $M \rightarrow R+R$
M		

Reduce-Reduce conflict

- Consider the grammar $M \rightarrow R + R \mid R + c \mid R$

$$R \rightarrow c$$

- input $c + c$

stack	input	action
	c+c	shift
c	+c	reduce $R \rightarrow c$
R	+c	shift
R+	c	shift
R+c		reduce $R \rightarrow c$
R+R		reduce $M \rightarrow R+R$
M		

stack	input	action
	c+c	shift
c	+c	reduce $R \rightarrow c$
R	+c	shift
R+	c	shift
R+c		reduce $M \rightarrow R+c$
M		